

Specialization for Coherence, Consistency, and Push/Pull for GPU Graph Analytics

Giordano Salvador¹, Wesley Darvin¹, Muhammad Huzaifa¹, John Alsop², Matthew Sinclair^{2,3}, Sarita Adve¹
University of Illinois at Urbana-Champaign¹, AMD Research², University of Wisconsin-Madison³

This work is supported in part by DARPA, NSF, a Google Faculty Research Award, and by the Applications Driving Architecture (ADA) Research center (JUMP center co-sponsored by SRC and DARPA)

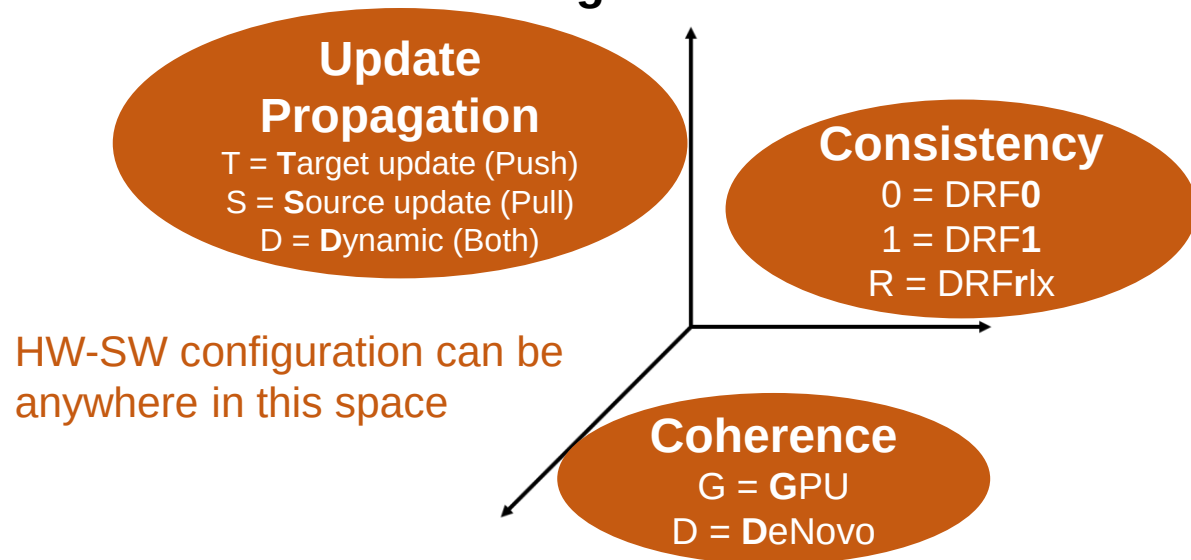
Motivation

- Specialization and heterogeneity key to efficiency
 - Memory system performance is critical
- Graph analytics increasingly important workload for CPU+GPU systems
 - Large diversity in graph algorithms and graph structures
- Key graph software specialization: Update propagation – Push vs. Pull?
 - Needs fine-grain synchronization for push, interacts with coherence and consistency
- This work: HW+SW specialization for graph analytics on GPU memory system
 - How to specialize update propagation, coherence, consistency?

Contributions

Specialize update propagation, coherence, consistency

Design Dimensions



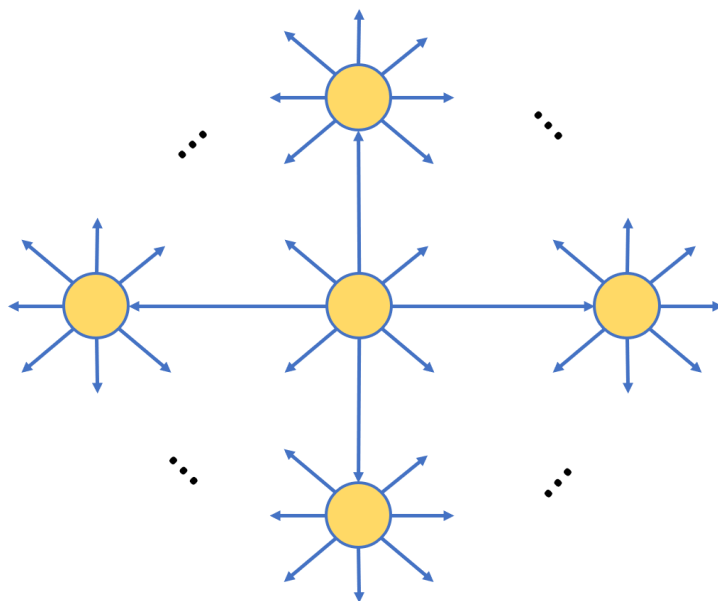
- Developed taxonomy and model to predict best configuration
- Evaluated 6 graph algorithms + 6 graph inputs = 36 workloads

Results

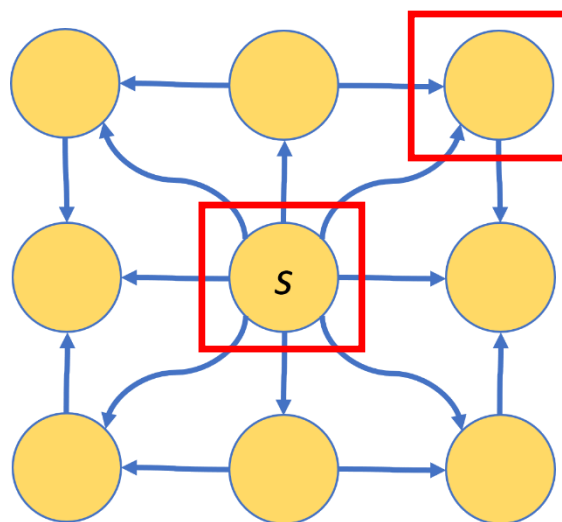
- One size does not fit all
 - 24/36 workloads: one config is best
 - For rest: other configs reduce execution time by 44% average, 87% max
- Model predicts best config for a workload
 - Predicts optimal config for 28/36 workloads
 - Within 3.5% of optimal for remaining
- Inter-dependent design dimensions
 - Choosing one dimension without considering others can give sub-optimal design
 - E.g., choosing Push over Pull for one workload
 - w/ DRFrIx support: 18% lower execution time
 - w/o DRFrIx support: 11% higher execution time

Taxonomy: Graph Structure

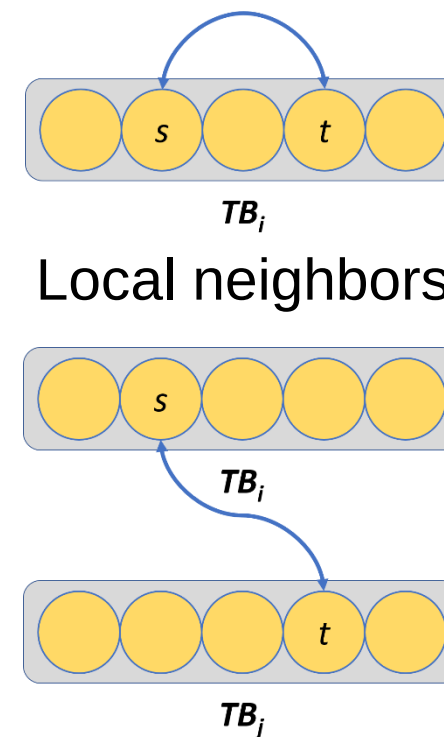
Volume



Imbalance



Reuse



Remote neighbors

Graph structure taxonomized as Volume, Imbalance, Reuse
Quantified as High, Medium, Low

Taxonomy: Algorithmic Properties

Traversal (AT)

Where graph
information is
propagated

Control (AC)

Whether graph
information is
propagated

Information (AI)

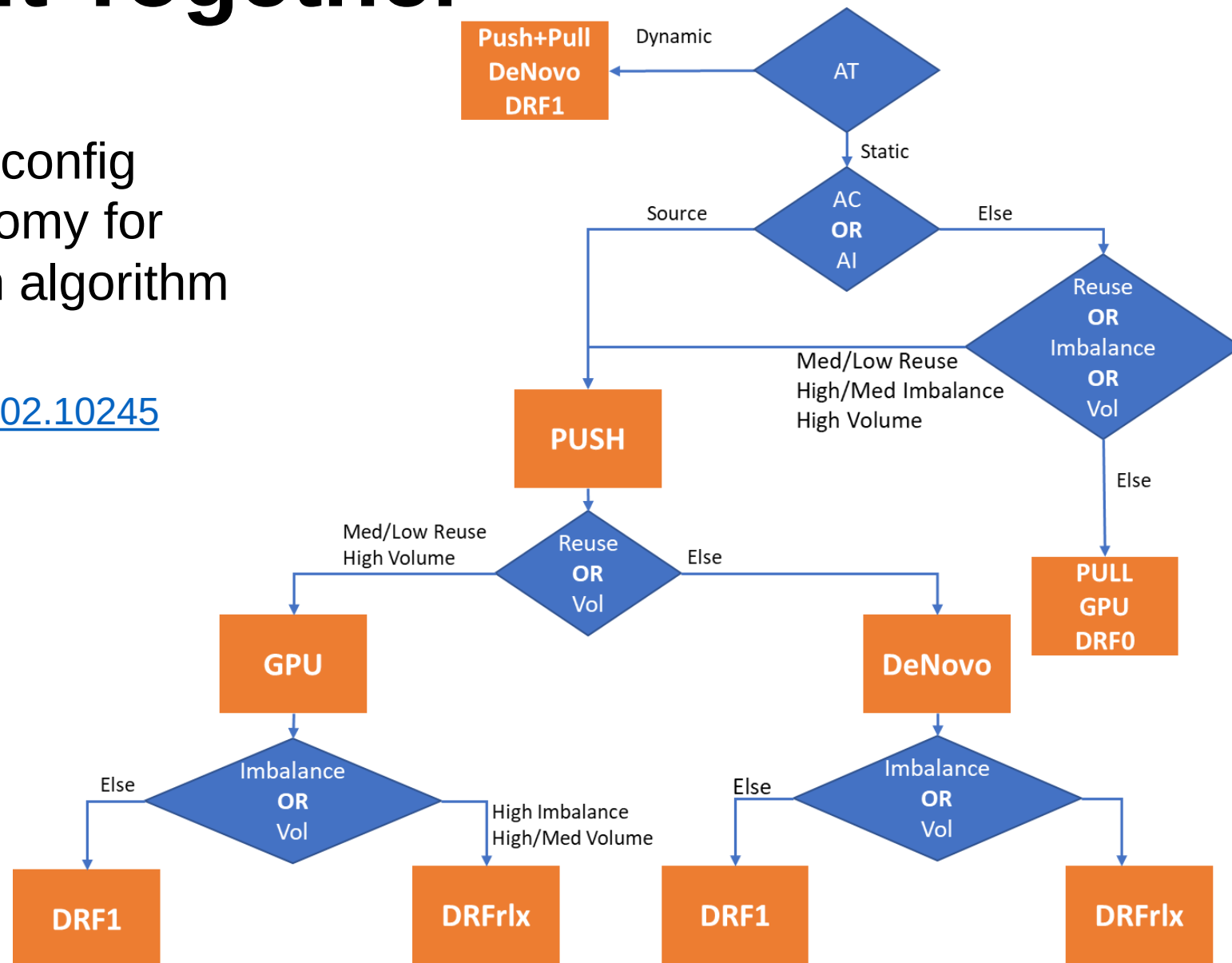
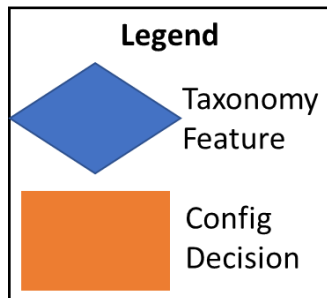
What graph
information is
propagated

```
PushKernel (V, E_in, E_out, op, v_prop)
  for src in V do
    if s_pred(src) then
      prop_s := v_prop(src)
      for edg in E_out(src) do
        tgt := dest(edg)
        if t_pred(tgt) then
          AtomicUpdate(op, tgt, prop_s)
```

AC and AI characterized as favoring source, target, or symmetric updates
AT characterized as either static or dynamic

Model – Putting it Together

- Goal: predict best HW+SW config
Build model using taxonomy for
Graph structure + Graph algorithm
- More info: <https://arxiv.org/abs/2002.10245>



Methodology

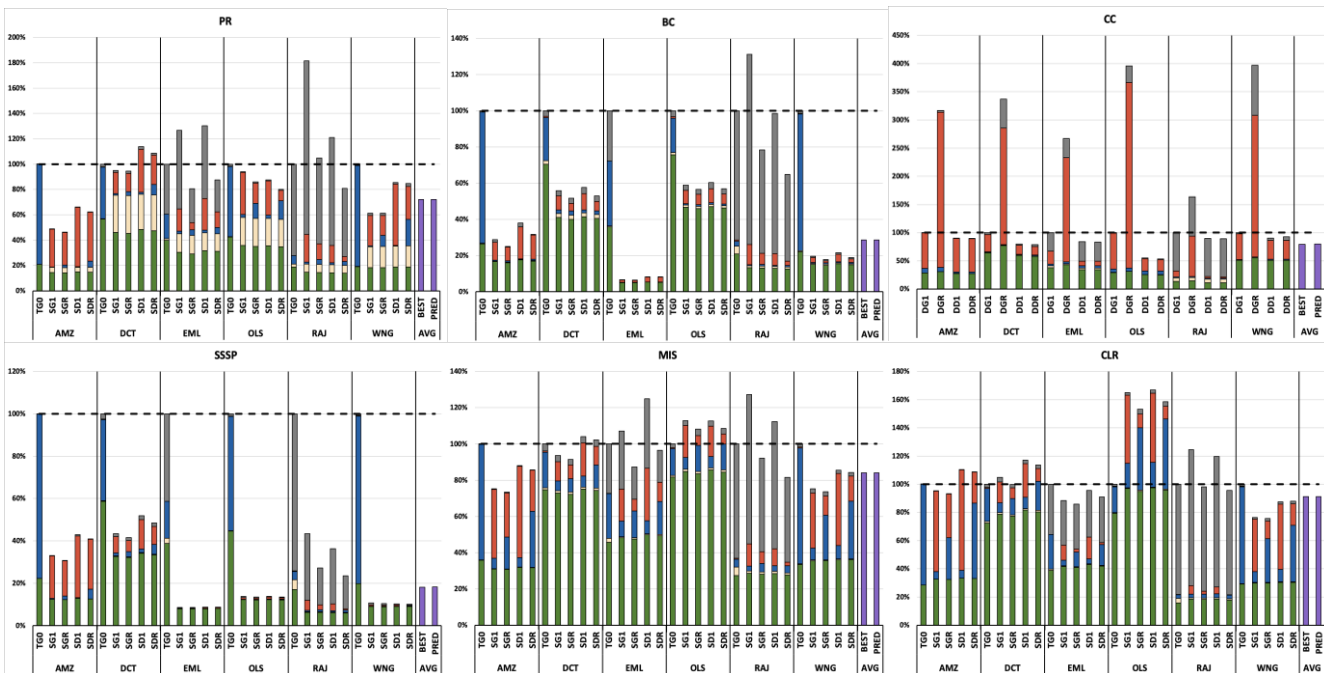
Graph	Volume	Locality	Imbalance
AMZ	High	Medium	Low
DCT	Medium	Medium	Medium
EML	High	Low	High
OLS	Medium	High	Low
RAJ	Low	High	High
WNG	Medium	Low	Low

Algorithm	Traversal	Control	Information
PR	Static	Symmetric	Source
SSSP	Static	Source	Source
MIS	Static	Symmetric	Symmetric
CLR	Static	Symmetric	Target
BC	Static	Source	Symmetric
CC	Dynamic w/ constricting races	-	-

- Evaluated 6 algorithms with 6 graphs for 36 workloads
- System Configuration
 - Integrated CPU-GPU SoC
 - 1 CPU core
 - 15 GPU compute units

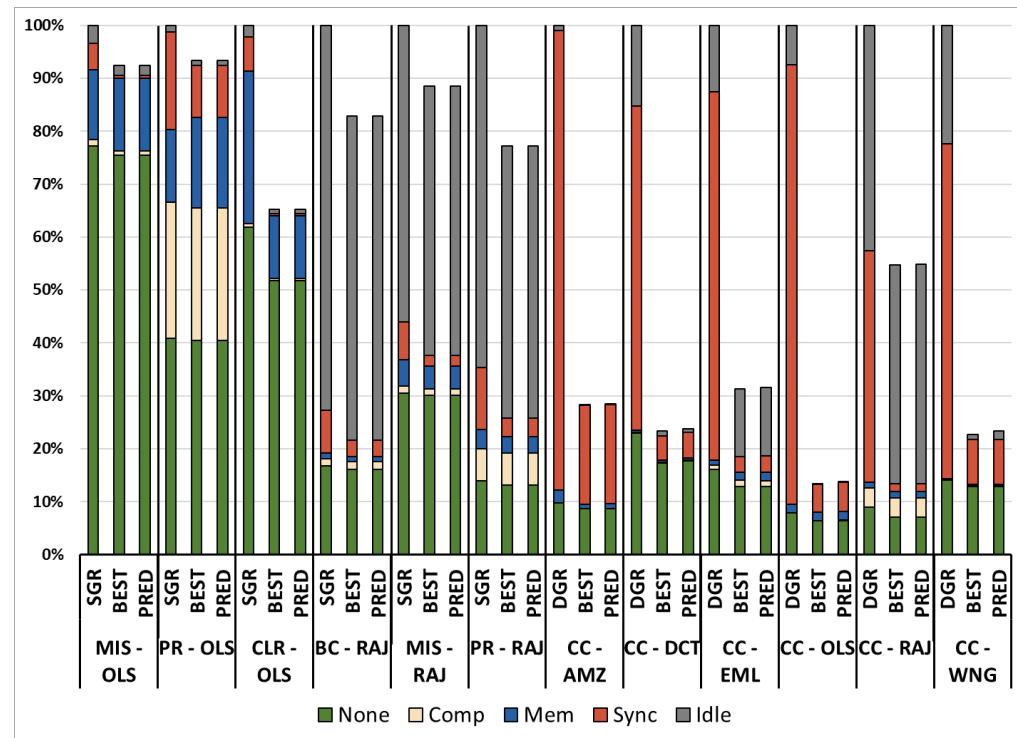
Results for Specialization

36 workloads across 5 HW/SW configurations



- Performance varies across algorithms, graphs, design space
- One configuration works best for 24/36 workloads
- BUT one size does not fit all

12 workloads where majority best config is not optimal



- Majority best config is worse than optimal for 12 cases
 - 44% average, 87% max
- One size does not fit all

Results for Model

	PR	SSSP	MIS	CLR	BC	CC
AMZ	SGR	SGR	SGR	SGR	SGR	<i>DD1</i>
DCT	SGR	SGR	SGR	SGR	SGR	<i>DD1</i>
EML	SGR	<i>SGR</i>	SGR	SGR	SGR	<i>DD1</i>
OLS	SDR	SDR	TG0	TG0	<i>SDR</i>	<i>DD1</i>
RAJ	SDR	SDR	SDR	SDR	SDR	<i>DD1</i>
WNG	SGR	<i>SGR</i>	SGR	SGR	SGR	DD1

Entry = Best configuration predicted by model
for algorithm + graph

Emphasized entries show mispredictions

Rest predicted correctly

- Model correctly predicts best HW/SW configuration for 28 of 36 workloads
- Mispredictions perform within 3.5% of best
- Mispredictions fall within our understanding of model
 - See arXiv paper for more information

Conclusion

- Specializing update propagation (synchronization), coherence, consistency for GPU graph analytics
 - Performance varies widely across graph algorithm, input, and design configuration
 - One size does not fit all
 - Majority best config is optimal for 24/36 workloads
 - 44% worse than optimal for 12/36 workloads on average (87% max)
 - Taxonomy and model to predict best configuration
 - Accurate prediction for 28/36 workloads
 - Remaining cases within 3.5% execution time of empirical best
- ⇒ Flexible software-hardware interfaces to specialize of coherence, consistency, synchronization
- Ongoing and future work
 - Adaptive runtime system
 - More specialized communication primitives
 - Flexible communication interfaces (e.g., Spandex [ISCA'18])